its knees. As the following listing shows, we can increase the replica count to increase the replica setting on the live index.

Listing 14.3 Increasing the replica count on a live index

```
PUT chats/_settings
{
  "number_of_replicas": 10
}
```

These additional replicas are per shard and are created on the newly formed nodes with data copied over to them. Now that we have additional replicas in our armory, they handle any read queries efficiently. It is Elasticsearch's responsibility to route client requests to the replicas, thus improving the application's search and query performance.

> **NOTE** A quick recap about resizing primary shards: once an index is created and live, we cannot resize it because `number_of_shards` is a static property of the index. If we must change this setting, we must *close* the index, create a new index with the new size, and *re-index* our data from the old index to our new index.

Although increasing read replicas increases read throughput, it strains the cluster's memory and disk space. That's because each replica consumes as many resources as its counterpart (primary shard).

14.2 Node communication

Elasticsearch hides the nitty-gritty details of what goes on behind the scenes, from starting the node to creating a cluster, indexing data, backups and snapshots, and querying. Adding nodes scales up the cluster and gives us the benefit of resilience from the start. We worked through a lot of APIs in earlier chapters, and those APIs enable communication between the client and the server. The expected communication medium in this case is an HTTP interface over RESTful APIs. The other side of the coin is communication between nodes: how each node talks to other nodes, how the master makes cluster-wide decisions, and so on. For this, Elasticsearch uses two types of communication:

- The HTTP interface for interactions between clients and nodes using RESTful APIs. (We looked at these when running queries.)
- The transport layer interface for node-to-node communications.

The cluster is exposed on port 9200 for HTTP (or HTTPS) communication by default, although we can change this by tweaking the configuration file (elasticsearch.yml). On the other hand, the transport layer is set to port 9300, meaning node-to-node communications happen on that port. Both interfaces are set in the configuration file for individual nodes under the `network` attribute, but we can change this based on our requirements.